Implementation Specification for Architecture-Aware Unified Optimization in Switch-Networked Quantum Data Centers

Internal LaTeX Design Document for Codex-Guided Experimental Development

March 23, 2026

Abstract

This document is an implementation specification, not a polished submission draft. It defines the target problem, modeling assumptions, optimization objective, constraints, algorithmic workflow, and evaluation plan for a new study on distributed quantum computing (DQC) over switch-networked quantum data centers (QDCs). The goal is to guide code generation and experiment development. The intended method inherits the unified optimization spirit of UNIQ while incorporating the architecture-aware mechanisms emphasized by switch-networked QDC systems: heterogeneous in-rack/cross-rack communication latency, switch reconfiguration overhead, buffer-aware scheduling, split-assisted cross-rack communication, collective in-rack EPR generation, and optional fidelity-aware distillation. This document should be treated by the coding agent as the source of truth unless the user explicitly overrides a design choice.

Contents

1 Purpose of This Document **2**

2 Introduction **3**
- 2.1 Target Research Question . . . 3
- 2.2 Scope of the First Implementation . . . 3

3 System Model **3**
- 3.1 QDC Architecture . . . 3
- 3.2 Circuit Model . . . 4
- 3.3 Time Model . . . 4

4 Problem Formulation **4**
- 4.1 Decision Variables . . . 4
- 4.2 Objective Function . . . 6
- 4.3 Constraint Set . . . 6
  - 4.3.1 Placement Feasibility . . . 6
  - 4.3.2 Gate Scheduling Feasibility . . . 6
  - 4.3.3 Communication Type Identification . . . 7
  - 4.3.4 Communication Mode Selection . . . 7
  - 4.3.5 EPR Availability and Consumption . . . 7
  - 4.3.6 Resource Capacity Constraints . . . 8
  - 4.3.7 Switch and Topology Constraints . . . 8
  - 4.3.8 Optional Fidelity Constraint . . . 9

5 Algorithmic Framework **9**
5.1 Design Principle . . . 9
5.2 Stage 1: Rack-Aware Greedy Placement . . . 10
5.3 Stage 2: Reconfiguration- and Buffer-Aware JIT Scheduling . . . 10
5.4 Stage 3: Direct-vs-Split Mode Selection . . . 11
5.5 Stage 4: Collective Generation . . . 11
5.6 Stage 5: Deadlock and Retry Handling . . . 11
5.7 Algorithm Pseudocode Skeleton . . . 11

6 Evaluation Plan **11**
6.1 Implementation Stack . . . 11
6.2 Default QDC Setting . . . 12
6.3 Benchmarks . . . 12
6.4 Baselines . . . 13
6.5 Ablations . . . 13
6.6 Metrics . . . 14
6.7 Core Experiments . . . 14

7 Implementation Plan for Codex **15**
7.1 Repository Structure . . . 15
7.2 Phased Coding Plan . . . 16
7.3 Sanity Checks . . . 16

8 Non-Negotiable Coding Requirements **16**

9 Expected Deliverables **17**

10 Final Instruction to the Coding Agent **17**

1 Purpose of This Document

This LaTeX file is meant to guide implementation. It should be readable like a short paper, but its real job is to keep the coding effort from degenerating into a toy simulator with pretty plots and no real model. The intended codebase must support:

- a switch-networked QDC simulator,
- a model-consistent optimization/scheduling framework,
- reproducible experiments with baselines and ablations,
- figure generation for representative examples, performance comparison, ablation, and scalability studies.

The implementation target is a *research-grade experimental framework*, not a classroom demo.

2 Introduction

Distributed quantum computing allows a large quantum circuit to be executed across multiple quantum processing units (QPUs), but remote inter-QPU interactions introduce substantial communication latency. In conventional DQC settings, the workflow is typically decomposed into three stages: qubit allocation, entanglement management, and network scheduling. A major lesson from prior unified optimization work is that these stages are strongly coupled, and optimizing them separately often fails to minimize total circuit runtime.

In switch-networked QDCs, the challenge becomes harder. The architecture is hierarchical and heterogeneous: communication within a rack is much faster than communication across racks, switch reconfiguration is non-negligible, prepared EPR pairs can be buffered only for limited periods, and flexible routing/splitting can improve latency at the cost of additional in-rack EPR consumption and potential fidelity degradation. Therefore, the new study should not merely transplant an old DQC algorithm onto a new topology. Instead, it should solve an *architecture-aware unified optimization* problem that jointly reasons about placement, communication mode selection, EPR preparation, switch configuration, buffering, and gate scheduling.

2.1 Target Research Question

Given a switch-networked QDC and a quantum circuit DAG, how should we jointly decide:

- where logical qubits are placed,
- whether each cross-rack interaction is served directly or via split-assisted communication,
- when in-rack and cross-rack EPR pairs are generated,
- when switch reconfiguration happens,
- how buffers are reserved and used,
- and when each gate is executed,

so that end-to-end runtime is minimized while communication overhead remains low and the schedule remains physically feasible?

2.2 Scope of the First Implementation

The first coding phase should prioritize runtime-oriented optimization with architecture-aware feasibility. Fidelity should be handled in a lightweight but explicit way, preferably as a hard threshold or post-hoc accounting metric rather than a full multi-objective optimization term. Distillation may be implemented as an optional extension after the core simulator is stable.

3 System Model

3.1 QDC Architecture

The QDC consists of a set of racks $\mathcal{R}$, a set of QPUs $\mathcal{U}$, and switch/network resources. Each QPU belongs to exactly one rack. We denote the rack of QPU u by $R(u) \in \mathcal{R}$.

Each QPU $u \in \mathcal{U}$ has:

- Cap_u: data-qubit capacity,

- E_u: number of communication qubits,
- Buf_u: number of buffer qubits available for storing prepared EPR pairs.

The architecture is hierarchical. Communication between two QPUs in the same rack is called *in-rack* communication. Communication between two QPUs in different racks is called *cross-rack* communication. We model the dominant communication-related latencies by:

$$L_{\text{intra}} > 0, \tag{1}$$
$$L_{\text{cross}} > L_{\text{intra}}, \tag{2}$$
$$L_{\text{rec}} > 0, \tag{3}$$

where L_{intra} is the time to generate an in-rack EPR pair, L_{cross} is the time to generate a cross-rack EPR pair, and L_{rec} is the switch reconfiguration latency.

3.2 Circuit Model

The quantum program is represented as a DAG of two-qubit gates:

- $\mathcal{Q}$: set of logical qubits,
- $\mathcal{G}$: set of two-qubit gates,
- $\mathcal{P} \subseteq \mathcal{G} \times \mathcal{G}$: precedence relations.

For each gate $g \in \mathcal{G}$, let (i_g, j_g) denote the pair of logical qubits involved in the gate.

The simulator should support both benchmark circuits loaded from Qiskit and synthetic DAGs constructed programmatically.

3.3 Time Model

We use a slotted time model for the optimization and simulation core. Let $\mathcal{T} = \{1, 2, \ldots, H\}$ denote the scheduling horizon. A convenient default is to choose the base slot equal to the in-rack communication unit, so that:

$$L_{\text{intra}} = 1 \text{ slot}, \tag{4}$$
$$L_{\text{rec}} = 10 \text{ slots}, \tag{5}$$
$$L_{\text{cross}} = 100 \text{ slots}, \tag{6}$$

which corresponds to the representative physical latency ratio used in the target QDC setting.

4 Problem Formulation

4.1 Decision Variables

The implementation should support the following decision variables or their algorithmic equivalents.

Placement variables.

$$\pi_{q,u} \in \{0, 1\}, \qquad \forall q \in \mathcal{Q},\ u \in \mathcal{U}, \tag{7}$$

where $\pi_{q,u} = 1$ if logical qubit q is assigned to QPU u.

Gate scheduling variables.

$$z_{g,t} \in \{0,1\}, \qquad \forall g \in \mathcal{G},\ t \in \mathcal{T}, \tag{8}$$

where $z_{g,t} = 1$ if gate g is executed at slot t.

Communication mode variables. For cross-rack interactions, we allow either direct realization or split-assisted realization:

$$x_g^{\text{dir}} \in \{0,1\}, \tag{9}$$

$$x_{g,w}^{\text{sp}} \in \{0,1\}, \qquad w \in \mathcal{W}_g, \tag{10}$$

where $\mathcal{W}_g$ is the candidate relay set for gate g.

EPR generation variables.

$$y_{g,t}^{\text{in}} \in \{0,1\}, \tag{11}$$

$$y_{g,t}^{\text{cr}} \in \{0,1\}, \tag{12}$$

$$y_{g,w,t}^{\text{post}} \in \{0,1\}, \tag{13}$$

representing, respectively, in-rack EPR generation, direct cross-rack EPR generation, and post-split in-rack EPR generation.

Resource state variables.

$$s_{ab,t}^{\text{in}}, s_{ab,t}^{\text{cr}} \in \mathbb{Z}_{\geq 0}, \tag{14}$$

represent the inventory of prepared in-rack and cross-rack EPR pairs for pair (a,b) after slot t.

Switch state variables.

$$c_{ab,t} \in \{0,1\}, \tag{15}$$

$$h_{s,t} \in \{0,1\}, \tag{16}$$

where $c_{ab,t} = 1$ indicates that a channel/path needed by pair (a,b) is active at slot t, and $h_{s,t} = 1$ indicates that switch s is reconfigured at slot t.

Buffer reservation variables.

$$rsv_{u,t} \in \mathbb{Z}_{\geq 0} \tag{17}$$

models buffer reservation used to avoid split-induced deadlock and future congestion.

4.2 Objective Function

To keep the model clean and implementation-oriented, the first version of the objective should remain close to the original unified DQC formulation:

$$\min \quad \alpha \sum_{g\in\mathcal{G}} \sum_{t\in\mathcal{T}} t\, z_{g,t} + \beta \sum_{g\in\mathcal{G}} \sum_{t\in\mathcal{T}} \sum_{u,v\in\mathcal{U}} \bar{C}_{u,v}\, \pi_{i_g,u} \pi_{j_g,v} z_{g,t}, \tag{18}$$

where $\alpha, \beta > 0$ are fixed coefficients and $\bar{C}_{u,v}$ is the rack-aware communication cost matrix:

$$\bar{C}_{u,v} = \begin{cases} 0, & u = v, \\ C_{\text{intra}}, & u \neq v,\ R(u) = R(v), \\ C_{\text{cross}}, & R(u) \neq R(v), \end{cases} \tag{19}$$

with $C_{\text{cross}} \gg C_{\text{intra}} > 0$.

The interpretation is straightforward:

- the first term encourages earlier gate completion and hence shorter total runtime,
- the second term discourages expensive communication patterns, especially cross-rack interactions.

Switch reconfiguration and fidelity effects should be modeled primarily through feasibility constraints and simulator accounting, rather than by adding too many extra weighted terms to the objective. That keeps the model implementable and avoids a parameter-tuning circus.

4.3 Constraint Set

The following constraints define the target model. The implementation can realize them either as explicit optimization constraints or as deterministic scheduling rules that enforce the same logic.

4.3.1 Placement Feasibility

C1. Unique assignment. Each logical qubit must be assigned to exactly one QPU:

$$\sum_{u\in\mathcal{U}} \pi_{q,u} = 1, \qquad \forall q \in \mathcal{Q}. \tag{20}$$

C2. QPU capacity. Each QPU has limited data-qubit capacity:

$$\sum_{q\in\mathcal{Q}} \pi_{q,u} \leq Cap_u, \qquad \forall u \in \mathcal{U}. \tag{21}$$

4.3.2 Gate Scheduling Feasibility

C3. Exactly-once execution. Each gate is executed exactly once:

$$\sum_{t\in\mathcal{T}} z_{g,t} = 1, \qquad \forall g \in \mathcal{G}. \tag{22}$$

C4. Precedence constraints. For each precedence edge $(g' \to g) \in \mathcal{P}$, the execution time of g must not precede that of g':

$$\sum_{\tau \le t} z_{g',\tau} \ge \sum_{\tau \le t} z_{g,\tau}, \qquad \forall (g' \to g) \in \mathcal{P},\ t \in \mathcal{T}. \tag{23}$$

4.3.3 Communication Type Identification

C5. Gate type induced by placement. Each gate is classified as local, same-rack remote, or cross-rack remote based on the selected placement. Define:

$$\delta_g^{\mathrm{sr}} = \sum_{\substack{u,v \in \mathcal{U} \\ u \ne v,\ R(u)=R(v)}} \pi_{i_g,u} \pi_{j_g,v}, \tag{24}$$

$$\delta_g^{\mathrm{cr}} = \sum_{\substack{u,v \in \mathcal{U} \\ R(u) \ne R(v)}} \pi_{i_g,u} \pi_{j_g,v}, \tag{25}$$

$$\delta_g^{\mathrm{loc}} = 1 - \delta_g^{\mathrm{sr}} - \delta_g^{\mathrm{cr}}. \tag{26}$$

This is a key upgrade from the old local/remote-only abstraction.

4.3.4 Communication Mode Selection

C6. Direct-vs-split mode selection. If a gate is cross-rack, it must choose exactly one realization mode:

$$x_g^{\mathrm{dir}} + \sum_{w \in \mathcal{W}_g} x_{g,w}^{\mathrm{sp}} = \delta_g^{\mathrm{cr}}, \qquad \forall g \in \mathcal{G}. \tag{27}$$

C7. Relay feasibility. A split relay must be physically feasible:

$$x_{g,w}^{\mathrm{sp}} \le A_{g,w}, \qquad \forall g \in \mathcal{G},\ w \in \mathcal{W}_g, \tag{28}$$

where $A_{g,w} \in \{0,1\}$ indicates whether relay w is allowed for gate g.

4.3.5 EPR Availability and Consumption

C8. EPR-before-gate rule for same-rack gates.

$$\sum_{\tau \le t} y_{g,\tau}^{\mathrm{in}} \ge z_{g,t} - (1 - \delta_g^{\mathrm{sr}}), \qquad \forall g, t. \tag{29}$$

C9. EPR-before-gate rule for direct cross-rack gates.

$$\sum_{\tau \le t} y_{g,\tau}^{\mathrm{cr}} \ge z_{g,t} - (1 - x_g^{\mathrm{dir}}), \qquad \forall g, t. \tag{30}$$

C10. EPR-before-gate rule for split-assisted gates. A split-assisted gate requires both the substitute cross-rack pair and the post-split in-rack pair:

$$\sum_{\tau \le t} y_{g,\tau}^{\mathrm{cr}} \ge z_{g,t} - (1 - x_{g,w}^{\mathrm{sp}}), \qquad \forall g, w, t, \tag{31}$$

$$\sum_{\tau \le t} y_{g,w,\tau}^{\mathrm{post}} \ge z_{g,t} - (1 - x_{g,w}^{\mathrm{sp}}), \qquad \forall g, w, t. \tag{32}$$

C11. EPR inventory recursion. For each pair (a, b) and resource type $\rho \in \{\text{in}, \text{cr}\}$:

$$s^{\rho}_{ab,t} = s^{\rho}_{ab,t-1} + \text{Gen}^{\rho}_{ab,t} - \text{Use}^{\rho}_{ab,t}, \tag{33}$$

with

$$s^{\rho}_{ab,0} = 0, \qquad s^{\rho}_{ab,t} \geq 0. \tag{34}$$

4.3.6 Resource Capacity Constraints

C12. Communication-qubit capacity. The number of active/stored EPRs touching a QPU cannot exceed available communication qubits:

$$\sum_{b \neq u} \left(s^{\text{in}}_{ub,t} + s^{\text{cr}}_{ub,t} \right) \leq E_u, \qquad \forall u, t. \tag{35}$$

C13. Buffer capacity. Including reserved future demand, the total buffer use cannot exceed buffer size:

$$\sum_{b \neq u} \left(s^{\text{in}}_{ub,t} + s^{\text{cr}}_{ub,t} \right) + rsv_{u,t} \leq Buf_u, \qquad \forall u, t. \tag{36}$$

C14. Deadlock-safe reservation. Reservation must cover active split pipelines:

$$rsv_{u,t} \geq \sum_{g,w} m^{\text{sp}}_{g,w,u} \, \xi_{g,w,t}, \qquad \forall u, t, \tag{37}$$

where $\xi_{g,w,t}$ indicates whether the split pipeline for (g, w) is still incomplete at slot t.

4.3.7 Switch and Topology Constraints

C15. Channel activation requirement. An EPR generation can occur only if the required channel/path is active:

$$y^{ab,\rho}_{g,t} \leq c_{ab,t}, \qquad \forall g, a, b, \rho, t. \tag{38}$$

C16. Switch capacity. Each switch or port group has a bounded number of simultaneous activations:

$$\sum_{(a,b) \in \Gamma(s)} c_{ab,t} \leq Cap^{\text{sw}}_{s}, \qquad \forall s, t. \tag{39}$$

C17. Reconfiguration indicator. If the channel configuration changes between adjacent slots, reconfiguration is triggered:

$$h_{s,t} \geq c_{ab,t} - c_{ab,t-1}, \tag{40}$$

$$h_{s,t} \geq c_{ab,t-1} - c_{ab,t}, \tag{41}$$

for all relevant $(a, b) \in \Gamma(s)$ and $t \geq 2$.

C18. Reconfiguration lockout. If $h_{s,t} = 1$, then switch s is unavailable for new EPR generation during the next L_{rec} slots. In code, this should be implemented as a stateful lockout rather than as a symbolic MILP-style big-M formulation.

C19. Collective generation capacity. If an in-rack pair (a, b) is active, multiple in-rack EPRs between the same QPUs can be generated in a batch:

$$\sum_{g} y_{g,t}^{ab,\text{in}} \leq M\, c_{ab,t}, \qquad \forall (a,b) \text{ same-rack},\ t, \tag{42}$$

where M is the batch upper bound.

4.3.8 Optional Fidelity Constraint

C20. Minimum fidelity. In the first implementation, fidelity may be enforced by a simple threshold:

$$F_g^{\text{dir}} x_g^{\text{dir}} + \sum_{w \in \mathcal{W}_g} F_{g,w}^{\text{sp}} x_{g,w}^{\text{sp}} \geq F_{\min}, \qquad \forall g \text{ with } \delta_g^{\text{cr}} = 1. \tag{43}$$

This can initially be implemented as a reporting metric and later upgraded to a strict feasibility filter.

5 Algorithmic Framework

5.1 Design Principle

Directly solving the full mixed discrete formulation at scale is unrealistic for the intended benchmarks. Therefore, the implementation should follow a constructive heuristic framework in the same spirit as the earlier unified DQC work, but upgraded for switch-networked QDCs.

The proposed framework consists of the following stages:

1. **Rack-aware greedy placement.** Place logical qubits onto QPUs by prioritizing heavily interacting pairs and minimizing expensive cross-rack edges.

2. **Reconfiguration-aware JIT scheduling.** Schedule gates as early as possible while respecting precedence, communication readiness, buffer limits, and switch lockout.

3. **Split-aware communication realization.** For a cross-rack demand, decide whether direct generation or split-assisted generation yields better expected completion time under current congestion.

4. **Collective in-rack batching.** Batch near-future in-rack EPR generations when they share the same QPU pair and channel activation.

5. **Optional distillation/fidelity enhancement.** For post-split in-rack EPRs, optionally allocate extra in-rack copies to improve effective fidelity.

5.2 Stage 1: Rack-Aware Greedy Placement

Construct an interaction graph over logical qubits, where edge weights count communication intensity between qubit pairs. The placement heuristic should:

- place strongly interacting qubits into the same QPU when possible,
- otherwise prefer the same rack over different racks,
- consider both QPU data-qubit capacity and anticipated communication pressure,
- optionally reserve communication-friendly racks for future hotspot qubits.

A practical first implementation is a greedy seed-and-grow mapping:

1. sort logical qubits or edges by descending interaction weight,
2. start from the heaviest pair,
3. iteratively assign the next unplaced qubit to the QPU/rack minimizing incremental cost,
4. break ties by remaining communication resources and current rack congestion.

5.3 Stage 2: Reconfiguration- and Buffer-Aware JIT Scheduling

Given a fixed placement, gates are scheduled in topological order. For each ready gate:

- if local, schedule at the earliest feasible slot,
- if same-rack remote, ensure an in-rack EPR is available or generate one, preferably by batching,
- if cross-rack remote, evaluate direct vs split modes using projected completion time and feasibility.

The scheduler should maintain time-evolving states for:

- QPU occupancy,
- communication-qubit availability,
- buffer occupancy,
- active channel configuration,
- switch lockout windows,
- EPR inventory and consumption.

5.4 Stage 3: Direct-vs-Split Mode Selection

For each cross-rack gate, compare:

- **Direct mode:** one cross-rack EPR generation subject to current endpoint/channel congestion,
- **Split mode:** one substitute cross-rack EPR plus one post-split in-rack EPR via a same-rack relay.

A simple but effective decision rule is:

$$\text{choose split if } \widehat{T}_{\text{split}} + \lambda_{\text{f}} \widehat{O}_{\text{fidelity}} < \widehat{T}_{\text{direct}}, \tag{44}$$

where $\widehat{T}_{\text{split}}$ and $\widehat{T}_{\text{direct}}$ are estimated completion times under current congestion, and $\widehat{O}_{\text{fidelity}}$ is a lightweight penalty or threshold indicator.

5.5 Stage 4: Collective Generation

The scheduler should inspect a short look-ahead window of ready or near-ready gates. If multiple in-rack EPR demands exist for the same QPU pair, it should batch them into one active period to amortize reconfiguration cost. This batching policy is a core architecture-aware feature and must not be omitted from the implementation.

5.6 Stage 5: Deadlock and Retry Handling

Flexible split and buffering can cause deadlock or pathological congestion. The implementation should therefore include:

- conservative reservation rules for future split completions,
- a deadlock detection condition based on stalled events and blocked resources,
- a retry mechanism that falls back to a more conservative scheduling choice when the aggressive plan becomes infeasible.

5.7 Algorithm Pseudocode Skeleton

6 Evaluation Plan

6.1 Implementation Stack

The first implementation should use:

- Python 3.11 or newer,
- Qiskit for reading benchmark circuits,
- NetworkX for DAG/topology representation,
- NumPy for core computation,
- Matplotlib for all figures,
- Pandas for tabulation and CSV logging.

Algorithm 1 Architecture-Aware Unified QDC Scheduling Framework

Require: circuit DAG $(\mathcal{Q}, \mathcal{G}, \mathcal{P})$, QDC architecture $(\mathcal{U}, \mathcal{R}, Cap, E, Buf)$, latency parameters, horizon H

Ensure: placement, communication schedule, gate schedule, performance statistics

```
Build weighted logical interaction graph
Compute rack-aware greedy placement π
Initialize simulator states: QPU usage, channel states, lockout states, EPR inventories, buffers
for gates in precedence-feasible order / event-driven ready queue do
    Identify gate type induced by π
    if gate is local then
        Schedule at earliest feasible slot
    else if gate is same-rack remote then
        Batch/generate in-rack EPR and schedule gate JIT
    else
        Evaluate direct vs split-assisted realization
        Reserve buffers if split is selected
        Schedule cross-rack and post-split EPR generations subject to switch/resource states
        Optionally attach distillation for post-split in-rack EPRs
        Schedule gate upon readiness
    end if
    if deadlock or persistent infeasibility is detected then
        Trigger retry with more conservative communication realization
    end if
end for
Collect metrics and generate logs/figures
```

6.2 Default QDC Setting

The default configuration for primary experiments should be:

- topology: CLOS,
- number of racks: 4,
- QPUs per rack: 4,
- data qubits per QPU: 30,
- communication qubits per QPU: 2,
- buffer qubits per QPU: 10,
- look-ahead depth: 10,
- slotized latencies: $L_{\text{intra}} = 1$, $L_{\text{rec}} = 10$, $L_{\text{cross}} = 100$.

6.3 Benchmarks

The benchmark suite should include:

- MCT,

- QFT,
- Grover,
- RCA,

and an additional synthetic workload family with the following stress patterns:

- dense cross-rack demand,
- dense in-rack burst demand,
- hotspot congestion,
- mixed random DAG.

6.4 Baselines

The implementation must support the following baselines.

B1. Random placement + earliest scheduling. A weak but necessary sanity baseline.

B2. Average / non-architecture-aware baseline. A simple method that ignores rack-aware structure and does not optimize switch reuse.

B3. On-demand direct-only baseline. No split, no collective generation, no distillation, no proactive buffer-aware scheduling.

B4. Optional SA-based placement baseline. Simulated annealing or another expensive metaheuristic for early synthetic comparison only.

6.5 Ablations

The main method should support feature toggles for the following ablations:

- No-Split,
- No-Collective,
- No-Distill,
- No-Reconfiguration-Aware,
- No-Buffer-Reservation,
- On-Demand-Only,
- Placement-Only,
- Scheduling-Only.

6.6 Metrics

At minimum, the framework must log:

- end-to-end latency / runtime,
- solver or compilation time,
- objective value from Eq. (18),
- latency breakdown: in-rack, cross-rack, reconfiguration,
- number of cross-rack EPR pairs,
- number of in-rack EPR pairs,
- number of distilled EPR pairs (if enabled),
- average EPR wait time,
- retry overhead,
- split ratio,
- peak and average buffer occupancy.

6.7 Core Experiments

The following experiments should be implemented in roughly this order.

E1. Representative example. A small hand-crafted case showing trivial scheduling, collective-only scheduling, split-only scheduling, and the full method. Output a Gantt-like figure with switch reconfiguration blocks, EPR generation blocks, and gate execution blocks.

E2. Synthetic workload comparison. Compare random, average, on-demand direct-only, and the full method on synthetic DAG families across multiple scales.

E3. Main benchmark comparison. Evaluate MCT/QFT/Grover/RCA under the default QDC architecture.

E4. Ablation study. Quantify the marginal effect of split, collective generation, buffer reservation, and reconfiguration-aware scheduling.

E5. Resource sensitivity. Vary communication qubits per QPU, buffer ratio, and look-ahead depth.

E6. Architecture sensitivity. Vary the number of racks, QPUs per rack, and network topology (CLOS, spine-leaf, fat-tree if implemented).

E7. Physical-parameter sensitivity. Vary reconfiguration latency and cross-rack latency ratios.

E8. Scalability study. Measure runtime and memory overhead as the number of algorithmic qubits and remote interactions grows.

7 Implementation Plan for Codex

7.1 Repository Structure

The generated repository should roughly follow this structure:

```
qdc_project/
  configs/
    default_qdc.yaml
    experiment_suites.yaml
  src/
    circuit/
      loaders.py
      synthetic_generators.py
      dag_utils.py
    topology/
      qdc_topology.py
      switch_model.py
    model/
      state.py
      metrics.py
      constraints.py
    algorithms/
      placement_greedy.py
      scheduler_baseline.py
      scheduler_unified.py
      split_policy.py
      batching_policy.py
      retry_policy.py
    simulation/
      engine.py
      events.py
      logger.py
    experiments/
      run_representative.py
      run_benchmarks.py
      run_ablation.py
      run_scaling.py
      run_sensitivity.py
    plotting/
      gantt.py
      barplots.py
      heatmaps.py
  outputs/
  tests/
```

`README.md`

7.2 Phased Coding Plan

Phase 1. Simulator core. Implement the QDC topology, time-stepped/event-driven simulation engine, EPR inventory tracking, switch lockout, and gate-DAG execution skeleton.

Phase 2. Minimal baselines. Implement random placement and direct-only on-demand scheduling. Produce at least one representative example figure and one benchmark CSV.

Phase 3. Architecture-aware placement and JIT scheduling. Implement the rack-aware greedy placement and the resource-aware earliest-feasible scheduler.

Phase 4. Split support. Implement relay selection, split bookkeeping, post-split in-rack generation, and entanglement-swapping completion logic.

Phase 5. Collective generation. Implement look-ahead batching and switch reuse.

Phase 6. Optional fidelity/distillation layer. Add distilled in-rack EPR generation and fidelity accounting.

Phase 7. Full experiment suite. Run all experiment families and generate publication-style figures.

7.3 Sanity Checks

The implementation must include automated checks for the following:

- every gate executes exactly once,
- no gate violates precedence,
- no QPU exceeds data-qubit capacity,
- no communication or buffer resource goes negative,
- no new EPR generation occurs during switch lockout,
- split completion consumes both required constituent pairs,
- all logs are reproducible under fixed random seeds.

8 Non-Negotiable Coding Requirements

The coding agent must not cut corners in the following ways:

- do not collapse the QDC model into a simple graph shortest-path abstraction,
- do not ignore switch reconfiguration state,

- do not implement split as a fake scalar penalty instead of an actual two-stage communication realization,
- do not replace collective generation with a post-hoc discount factor,
- do not output figures from fabricated placeholder data,
- do not place all logic into one giant script.

9 Expected Deliverables

The first complete code delivery should include:

- a runnable repository,
- reproducible configuration files,
- one representative-example script,
- one benchmark-comparison script,
- unit tests for simulator invariants,
- generated CSV logs and figures,
- a concise README explaining how to reproduce experiments.

10 Final Instruction to the Coding Agent

Treat this document as the implementation specification and source of truth. Start from a faithful simulator and minimal baselines, then layer in architecture-aware features one by one. A correct, extensible experimental framework is more important than prematurely optimizing for elegance or brevity.